

crop yield prediction

June 24, 2025

1 Crop Yield Prediction

The aim of this notebook is to predict the crop yield using the data from the given dataset. The dataset has been taken from the [Crop Yield Prediction](#).

1.0.1 Data Dictionary

Column Name	Description
Rain Fall (mm)	Rainfall in millimeters
Temperature (C)	Temperature in Celsius
Fertilizer (kg)	Fertilizer in kilograms
Nitrogen (N)	Nitrogen macro nutrient
Phosphorous (P)	Phosphorous macro nutrient
Potassium (K)	Potassium macro nutrient
Yield (Q/acres)	Crop yield Quintals per acre

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[2]: df = pd.read_excel("crop yield data sheet.xlsx")
df.head()
```

```
[2]:
```

	Rain Fall (mm)	Fertilizer	Temperatue	Nitrogen (N)	Phosphorus (P)	\
0	1230.0	80.0	28	80.0	24.0	
1	480.0	60.0	36	70.0	20.0	
2	1250.0	75.0	29	78.0	22.0	
3	450.0	65.0	35	70.0	19.0	
4	1200.0	80.0	27	79.0	22.0	

	Potassium (K)	Yeild (Q/acre)
0	20.0	12.0
1	18.0	8.0
2	19.0	11.0
3	18.0	9.0
4	19.0	11.0

1.1 Data Preprocessing

```
[3]: #Checking the shape of the dataset
df.shape
```

```
[3]: (109, 7)
```

```
[4]: #Checking the data types of the columns
df.dtypes
```

```
[4]: Rain Fall (mm)    float64
      Fertilizer      float64
      Temperatue      object
      Nitrogen (N)    float64
      Phosphorus (P)  float64
      Potassium (K)   float64
      Yeild (Q/acre)  float64
      dtype: object
```

Here, the temperature has object data type. We need to convert it to float data type. But first, I am checking the values in the column

```
[5]: df['Temperatue'].unique()
```

```
[5]: array([28, 36, 29, 35, 27, 34, 37, 39, 26, 38, 24, 25, 40, nan, ':'],
      dtype=object)
```

The column has invalid value ":" .I need to remove this value before converting the column to float data type.

```
[6]: #Dropping ":" from the Temperatue column
df = df[df['Temperatue'] != ':']
```

```
[7]: #converting the Temperatue column to float
df['Temperatue'] = df['Temperatue'].astype(float)
```

```
[8]: #Checking for null values
df.isnull().sum()
```

```
[8]: Rain Fall (mm)    9
      Fertilizer      9
      Temperatue      9
      Nitrogen (N)    9
      Phosphorus (P)  9
      Potassium (K)   9
      Yeild (Q/acre)  9
      dtype: int64
```

Replacing missing values with median of the column

```
[9]: #replacing missing values with median
columns = [df.columns]
for col in columns:
    df[col] = df[col].fillna(df[col].median())
```

Descriptive Statistics

```
[10]: df.describe()
```

```
[10]:
```

	Rain Fall (mm)	Fertilizer	Temperatue	Nitrogen (N)	Phosphorus (P)	\
count	108.000000	108.000000	108.000000	108.000000	108.000000	
mean	874.814815	67.990741	32.111111	70.759259	21.120370	
std	391.818744	9.616473	5.277944	6.390516	1.868167	
min	400.000000	50.000000	24.000000	59.000000	18.000000	
25%	450.000000	60.000000	28.000000	65.000000	20.000000	
50%	1150.000000	70.000000	29.000000	71.000000	21.000000	
75%	1226.250000	77.000000	38.000000	76.250000	23.000000	
max	1300.000000	80.000000	40.000000	80.000000	25.000000	

	Potassium (K)	Yeild (Q/acre)
count	108.000000	108.000000
mean	18.138889	9.046296
std	1.758601	1.881460
min	15.000000	5.500000
25%	16.000000	7.000000
50%	19.000000	9.000000
75%	19.000000	11.000000
max	22.000000	12.000000

```
[11]: df.head()
```

```
[11]:
```

	Rain Fall (mm)	Fertilizer	Temperatue	Nitrogen (N)	Phosphorus (P)	\
0	1230.0	80.0	28.0	80.0	24.0	
1	480.0	60.0	36.0	70.0	20.0	
2	1250.0	75.0	29.0	78.0	22.0	
3	450.0	65.0	35.0	70.0	19.0	
4	1200.0	80.0	27.0	79.0	22.0	

	Potassium (K)	Yeild (Q/acre)
0	20.0	12.0
1	18.0	8.0
2	19.0	11.0
3	18.0	9.0
4	19.0	11.0

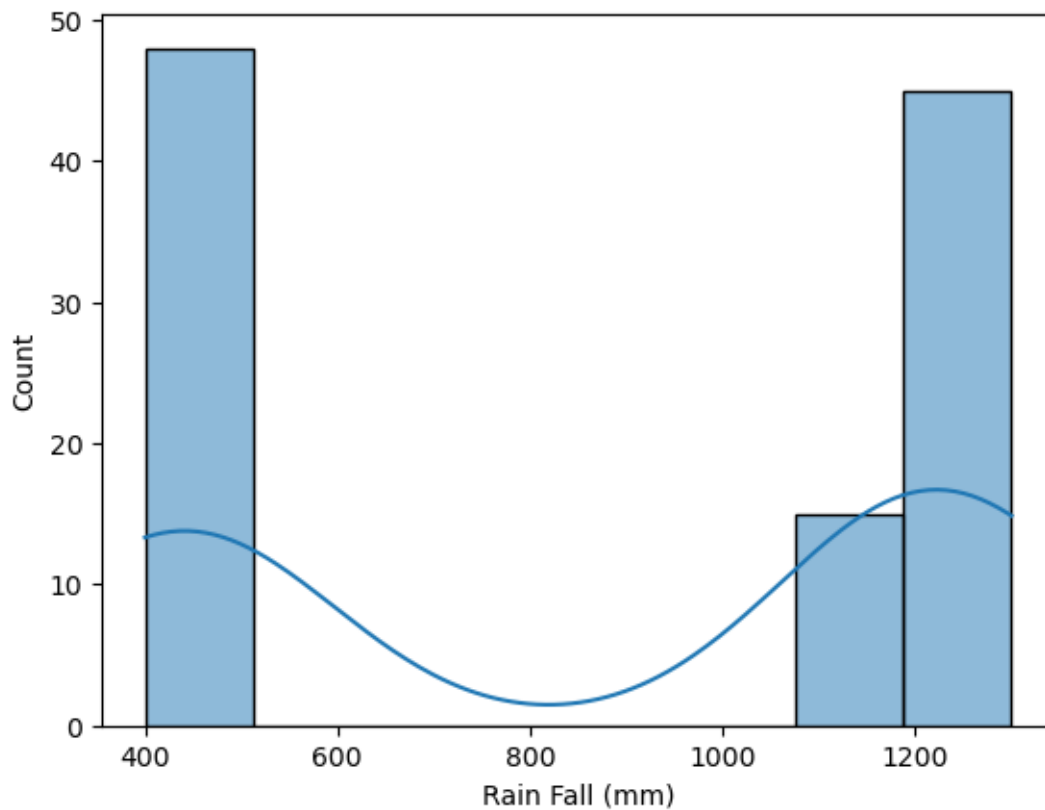
1.2 Exploratory Data Analysis

1.2.1 Rainfall Distribution

```
[12]: sns.histplot(x = "Rain Fall (mm)", data = df, kde = True)
```

```
C:\Users\jeane\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119:  
FutureWarning: use_inf_as_na option is deprecated and will be removed in a  
future version. Convert inf values to NaN before operating instead.  
with pd.option_context('mode.use_inf_as_na', True):
```

```
[12]: <Axes: xlabel='Rain Fall (mm)', ylabel='Count'>
```



In the above histogram we can see the distribution of the rainfall in mm. The graph reveals that the rainfall is not normally distributed. Either the rainfall is between 400 - 500 mm or it is above 1100 mm. This could indicate that the dataset has been collected for different crops which require different amount of rainfall.

1.2.2 Fertilizer Distribution

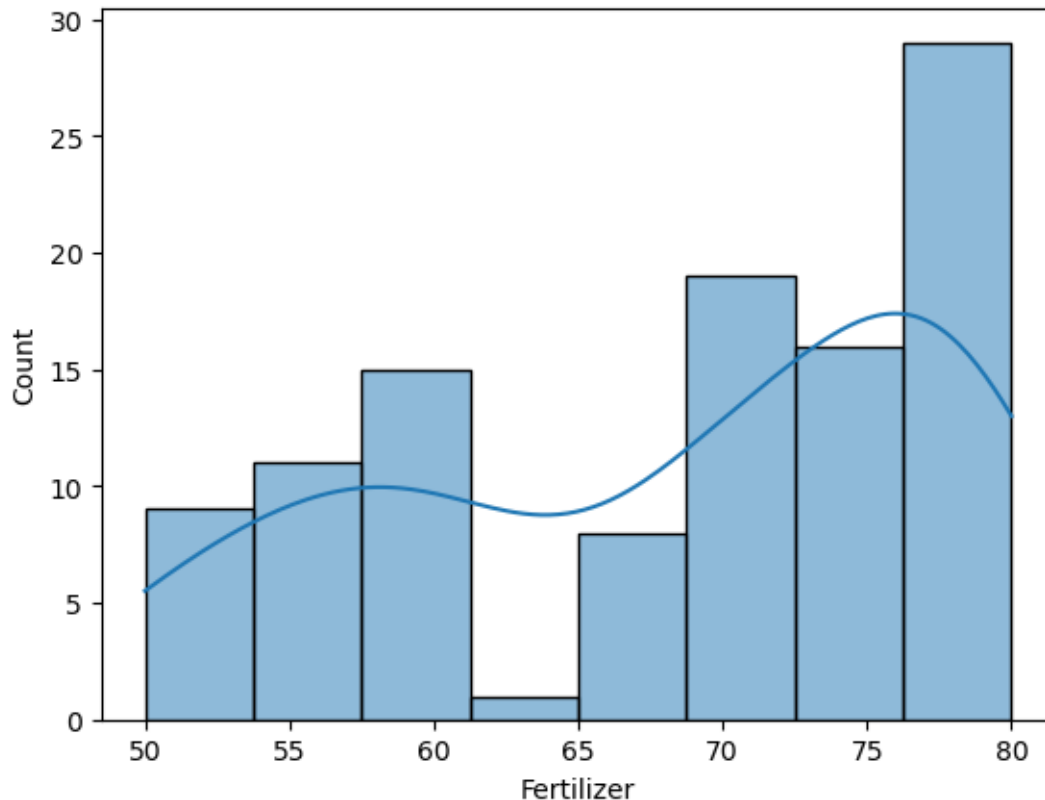
```
[13]: sns.histplot(x = "Fertilizer", data = df, kde = True)
```

```
C:\Users\jeane\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119:
```

FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.

```
with pd.option_context('mode.use_inf_as_na', True):
```

```
[13]: <Axes: xlabel='Fertilizer', ylabel='Count'>
```



This graph shows the distribution of fertilizer used for the crop. In the graph we can see there are two divisions in the usage of fertilizer. One is less than 65 and the other is above 65. Fertilizer has huge impact on the crop yield. Therefore, we can assume that there could be a proportional relationship between the fertilizer and the crop yield. We can confirm this assumption in the latter section of the EDA.

1.2.3 Temperature Distribution

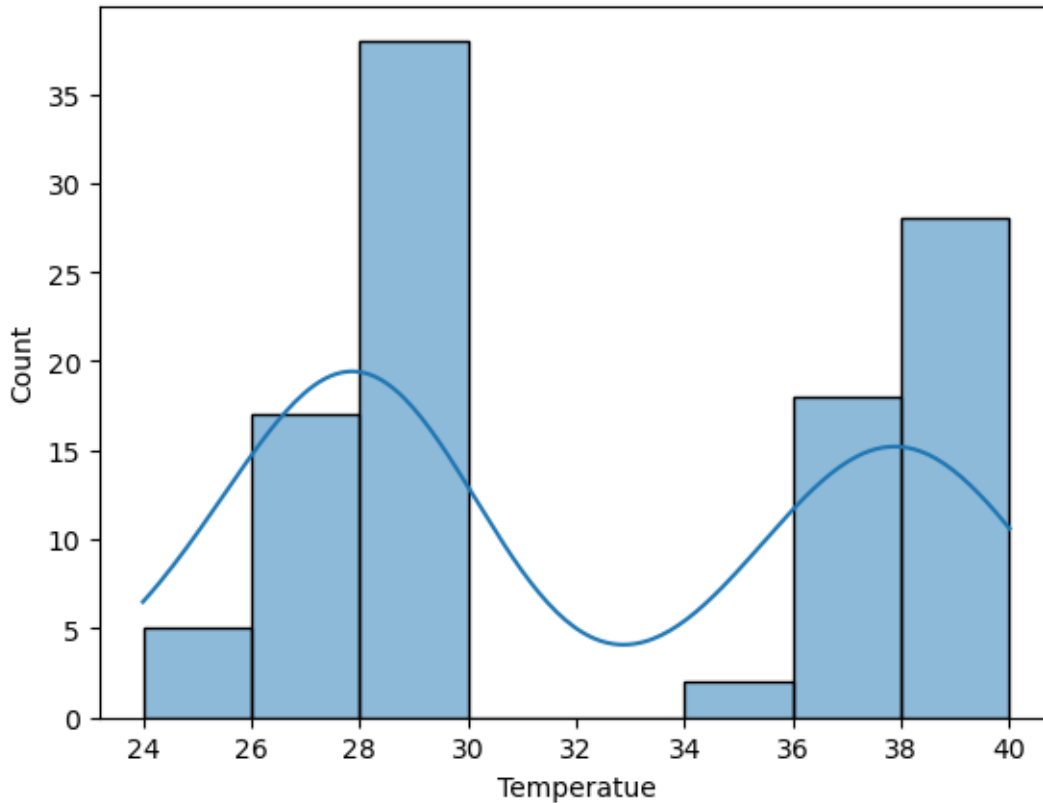
```
[14]: sns.histplot(x="Temperatue", data = df, kde = True)
```

C:\Users\jeane\anaconda3\Lib\site-packages\seaborn_oldcore.py:1119:

FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.

```
with pd.option_context('mode.use_inf_as_na', True):
```

```
[14]: <Axes: xlabel='Temperatue', ylabel='Count'>
```



The temperature graph also shows two similar peaks just like the pervious two graphs, which hints that the dataset could be collected for two different crops. The temperature is not normally distributed. The temperature is either between 25 - 30 or 35 - 40. This may indicate that their could be a rabi and kharif crop in the dataset.

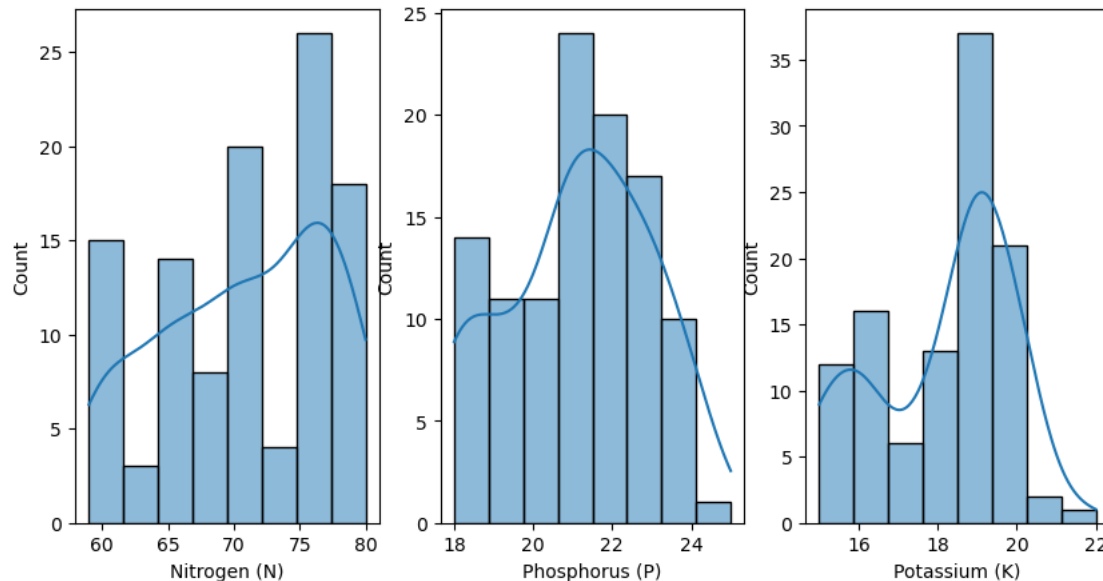
1.2.4 Macronutrients (NPK) Distribution

```
[15]: fig, ax = plt.subplots(1,3,figsize=(10, 5))
sns.histplot(x = "Nitrogen (N)", data = df, kde = True, ax = ax[0])
sns.histplot(x = "Phosphorus (P)", data = df, kde = True, ax = ax[1])
sns.histplot(x = "Potassium (K)", data = df, kde = True, ax = ax[2])
```

```
C:\Users\jeane\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
future version. Convert inf values to NaN before operating instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Users\jeane\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
future version. Convert inf values to NaN before operating instead.
    with pd.option_context('mode.use_inf_as_na', True):
C:\Users\jeane\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
```

```
future version. Convert inf values to NaN before operating instead.
with pd.option_context('mode.use_inf_as_na', True):
```

```
[15]: <Axes: xlabel='Potassium (K)', ylabel='Count'>
```



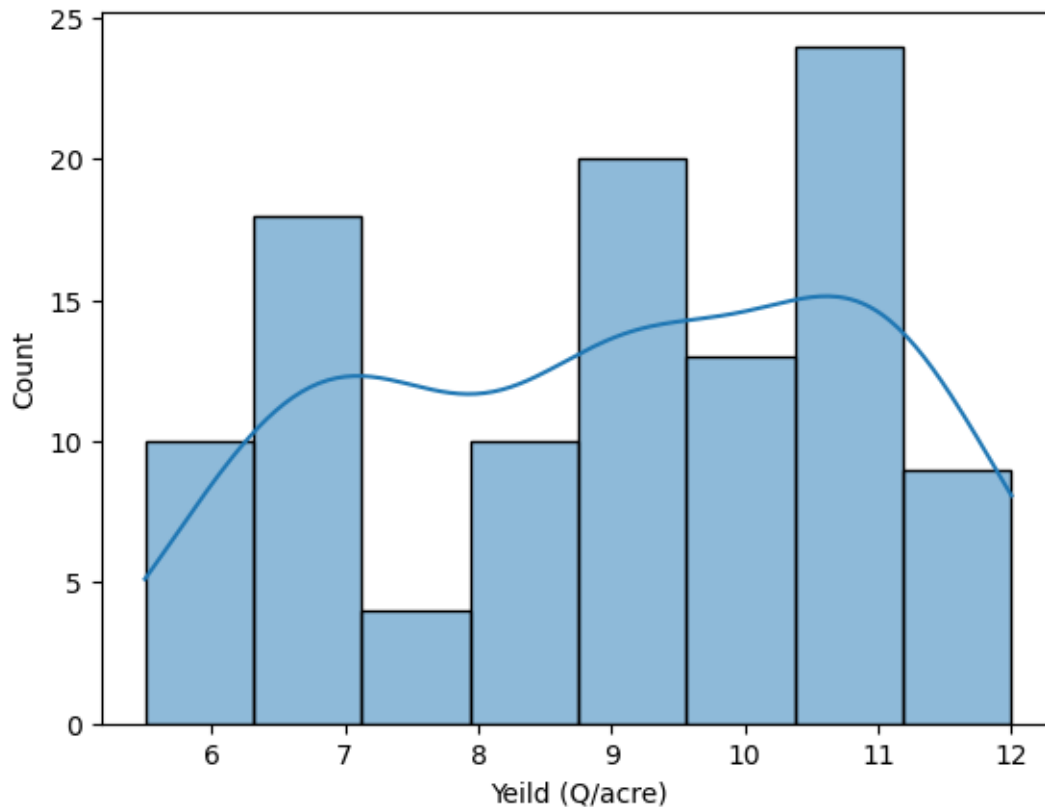
These three graphs shows that the distribution of the macronutrients used in the crop. From the graph for Nitrogen, we can see that larger quantity of nitrogen is used in the crop. The graph for Phosphorous shows that the usage of phosphorous is less than nitrogen. The graph for Potassium shows that the usage of potassium is very less compared to nitrogen and phosphorous but still it has two peaks. This could indicate that the dataset has been collected for two different crops with different potassium requirements but similar nitrogen and phosphorous requirements.

1.2.5 Yield Distribution

```
[16]: sns.histplot(x = "Yeild (Q/acre)", data = df, kde = True)
```

```
C:\Users\jeane\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
future version. Convert inf values to NaN before operating instead.
with pd.option_context('mode.use_inf_as_na', True):
```

```
[16]: <Axes: xlabel='Yeild (Q/acre)', ylabel='Count'>
```



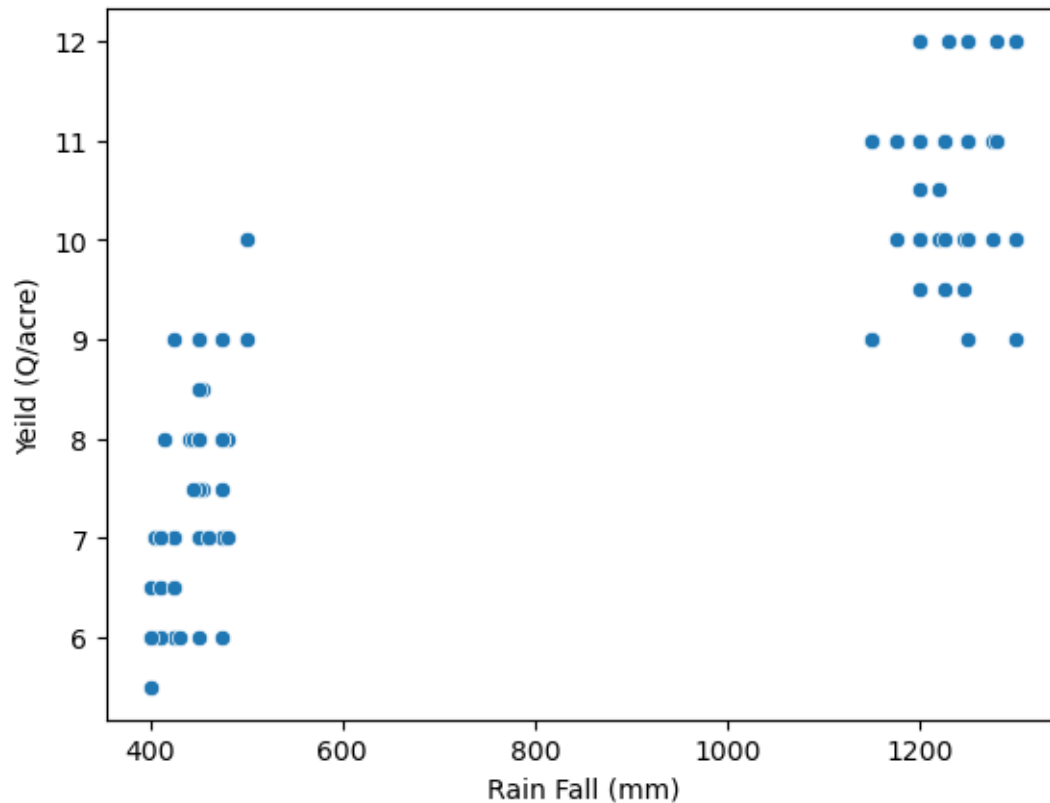
The graph shows the distribution of the crop yield. The graph has two smaller peaks which could indicate the difference in the yield of two different crops on the same kind of soil.

Now, after reviewing the distribution of data across all the columns, I have made some hypothesis such as presence of two different crops in the dataset, and the relationship between the crop yield and the other columns. Now, I will test these hypothesis in the next section.

1.2.6 RainFall and Crop Yield

```
[17]: sns.scatterplot(x = 'Rain Fall (mm)', y = 'Yeild (Q/acre)', data = df)
```

```
[17]: <Axes: xlabel='Rain Fall (mm)', ylabel='Yeild (Q/acre)'\>
```

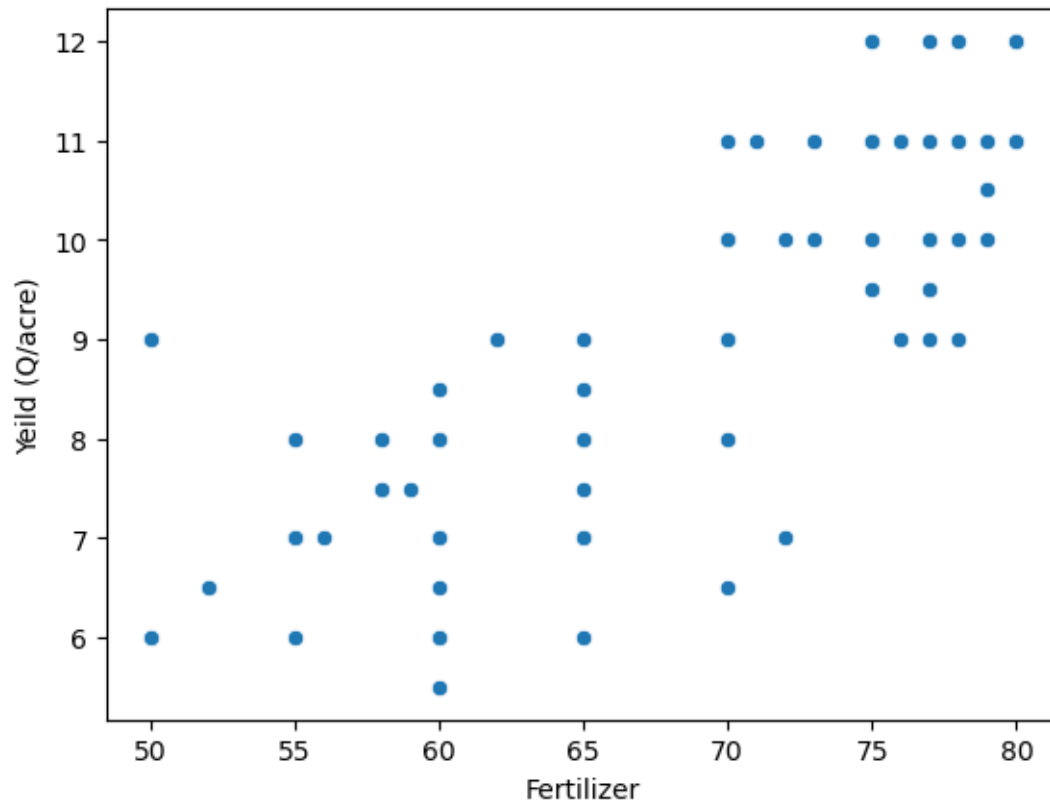



In this graph we can clearly see two clusters of data. The first cluster is between 400 - 500 mm and the second cluster is above 1100 mm. This shows that one crop has lower rainfall requirement and the other has higher rainfall requirement. However, still there is difference in the yield of the crops, which could be due to the other factors such as temperature, fertilizer, and macronutrients and most importantly the type of soil.

1.2.7 Ferilizer and Crop Yield

```
[18]: sns.scatterplot(x = 'Fertilizer', y = 'Yeild (Q/acre)', data = df)
```

```
[18]: <Axes: xlabel='Fertilizer', ylabel='Yeild (Q/acre)'>
```

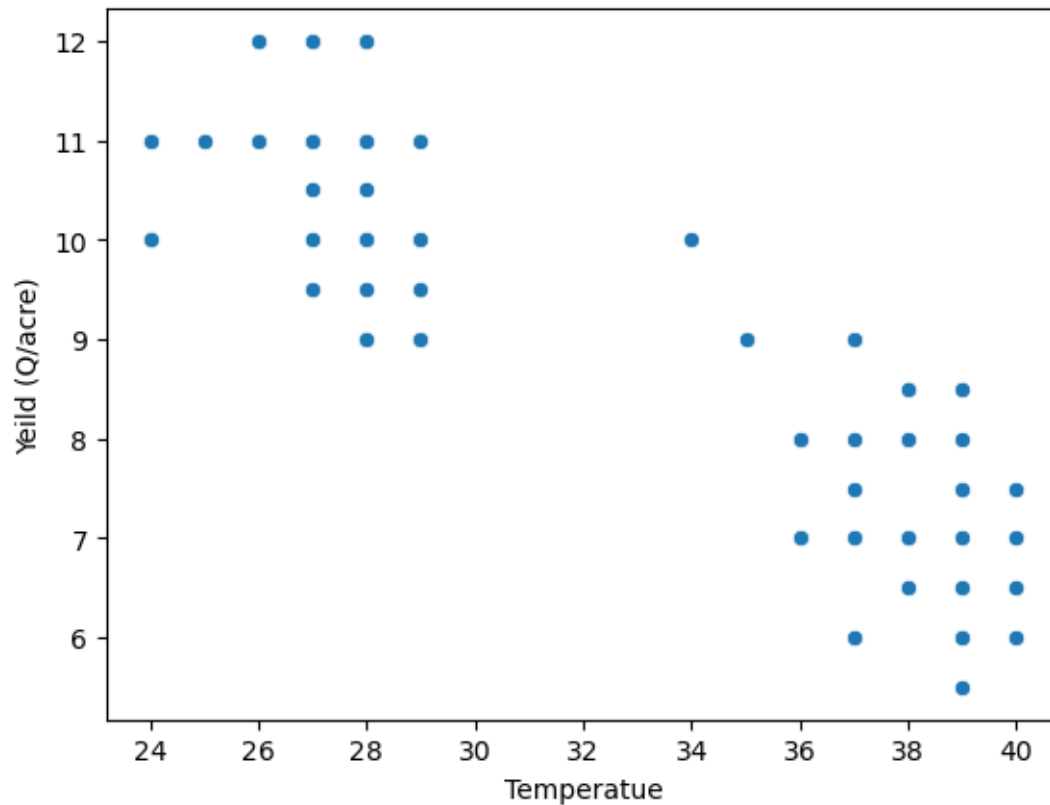


This graph shows that the crop yield is not directly proportional to the amount of fertilizer used. There are many data points where the crop yield is high even when the fertilizer used is less than 65. This could indicate that the type of soil and the macronutrients in the soil could be the reason for the high yield.

1.2.8 Temperature and Crop Yield

```
[19]: sns.scatterplot(x = 'Temperature', y = 'Yield (Q/acre)', data = df)
```

```
[19]: <Axes: xlabel='Temperature', ylabel='Yield (Q/acre)'>
```

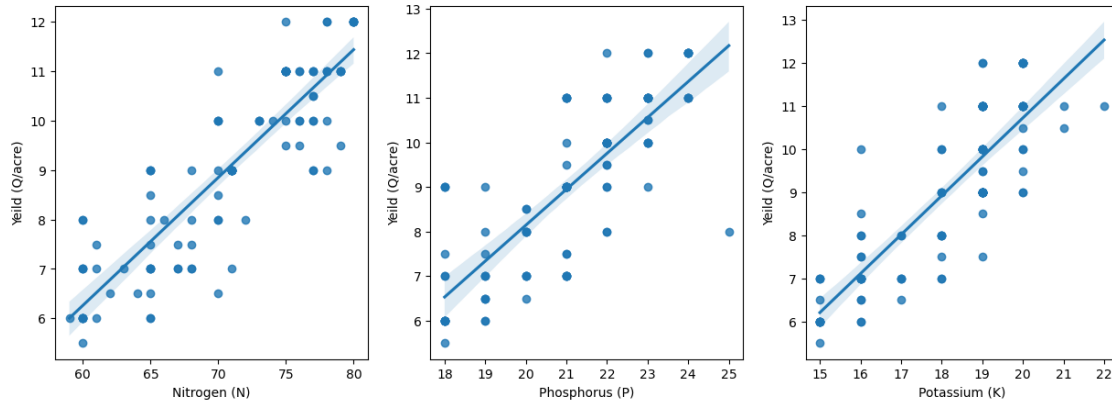


The graph of temperature and crop yield also shows two clusters. The first cluster is between 25 - 30 and the second cluster is between 35 - 40. This could indicate that the dataset has been collected for two different crops. In the first cluster, the crop yield is higher than the second cluster. This could indicate that the first cluster could be the rabi crop and the second cluster could be the kharif crop.

1.2.9 Macronutrients and Crop Yield

```
[20]: fig, ax = plt.subplots(1,3,figsize=(15, 5))
sns.regplot(x = 'Nitrogen (N)', y = 'Yeild (Q/acre)', data = df, ax = ax[0])
sns.regplot(x = 'Phosphorus (P)', y = 'Yeild (Q/acre)', data = df, ax = ax[1])
sns.regplot(x = 'Potassium (K)', y = 'Yeild (Q/acre)', data = df, ax = ax[2])
```

```
[20]: <Axes: xlabel='Potassium (K)', ylabel='Yeild (Q/acre)'\>
```

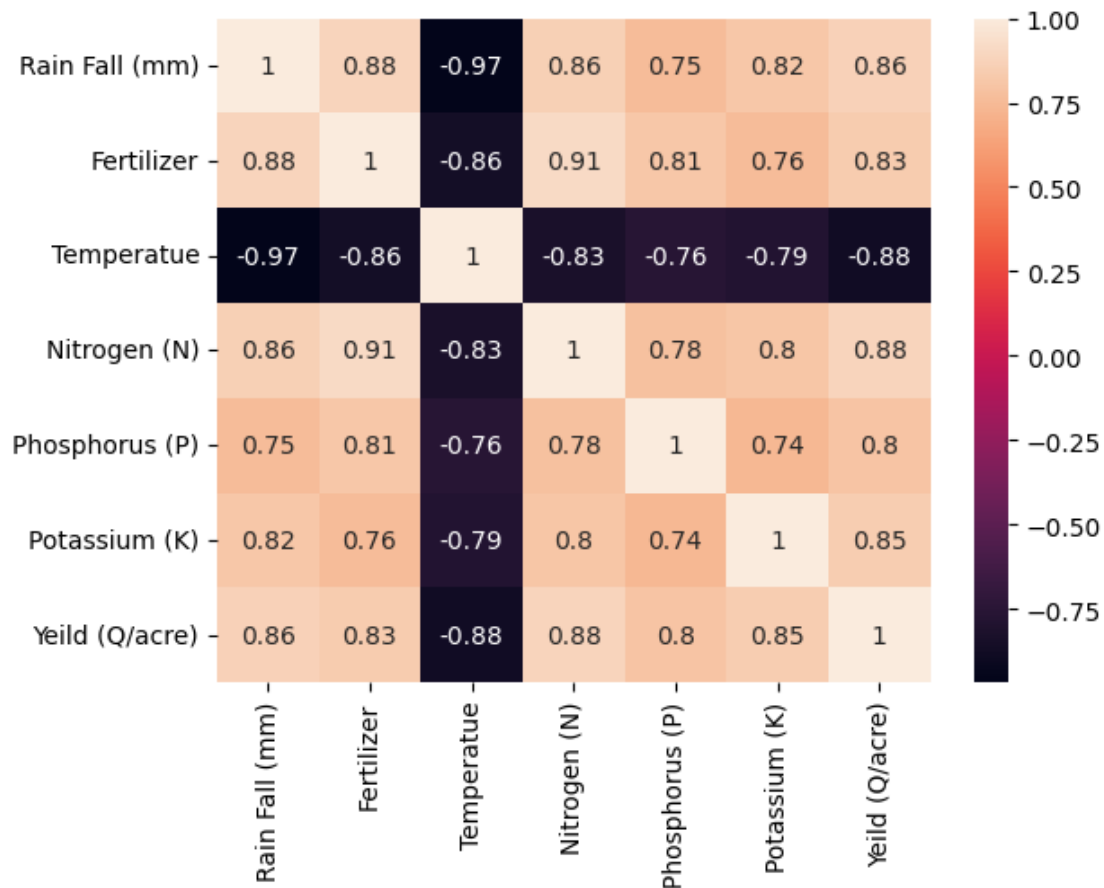


Here in the graphs between the nutrients and the crop yield, we can see a hint of proportional relationship between the nutrients and the crop yield. The crop yield is higher when the nutrients are higher. This could indicate that the soil has less nutrients and the crop yield is less. However, still there are many data points which deviate from the proportional relationship. This could be due to other miscellaneous factors such as crop breed, soil type, and weather conditions.

1.2.10 Correlation Matrix Heatmap

```
[21]: sns.heatmap(df.corr(), annot = True)
```

```
[21]: <Axes: >
```



From the above EDA, we can see that the dataset has been collected for two different crops. The dataset has two clusters for rainfall, temperature, and crop yield. There are signs of proportional relationship between the nutrients and the crop yield. However, the relationship between the crop yield and the other columns is not directly proportional. This could be due to the other factors such as soil type, weather conditions, and crop breed. On the whole, the dataset is not much complex to make concrete decisions based on the graphs only.

1.3 Train Test Split

```
[36]: from sklearn.model_selection import KFold, cross_val_score
# Define features and target
X = df.drop('Yeild (Q/acre)', axis=1)
y = df['Yeild (Q/acre)']

# Set up the same K-Fold cross-validator for both models
cv = KFold(n_splits=5, shuffle=True, random_state=42)
```

1.4 Model Building

I will be using the following models to predict the crop yield:

1. Decision Tree Regressor
2. Random Forest Regressor

1.4.1 Decision Tree Regressor

```
[37]: from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import KFold, cross_validate
#Decision Tree Regressor object
# Initialize models
dt_model = DecisionTreeRegressor(random_state=42)
rf_model = RandomForestRegressor(random_state=42)
rf_top3 = RandomForestRegressor(random_state=42)
```

```
[38]: # Entraînement du modèle non optimisé
rf_model.fit(X, y)

# Extraction des 3 features les plus importantes
feature_importances = pd.Series(
    rf_model.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

top3_features = feature_importances.head(3).index.tolist()
print("Top 3 features :", top3_features)
```

Top 3 features : ['Rain Fall (mm)', 'Temperatue', 'Nitrogen (N)']

Definition des metrics and Cross validation

```
[40]: from sklearn.metrics import make_scorer, mean_absolute_error,
↳ mean_squared_error, r2_score
# Define multiple scoring metrics
scoring = {
    'r2': 'r2',
    'mae': 'neg_mean_absolute_error',
    'mse': 'neg_mean_squared_error'
}

# Cross-validation
dt_cv = cross_validate(dt_model, X, y, cv=cv, scoring=scoring,
↳ return_train_score=False)
rf_cv = cross_validate(rf_model, X, y, cv=cv, scoring=scoring,
↳ return_train_score=False)
rf_top3_cv = cross_validate(rf_top3, X[top3_features], y, cv=cv,
↳ scoring=scoring, return_train_score=False)
```

```

# Function to format results
def summarize_results(cv_result, model_name):
    mae = -cv_result['test_mae']
    mse = -cv_result['test_mse']
    rmse = np.sqrt(mse)
    r2 = cv_result['test_r2']

    summary = {
        'Model': model_name,
        'MAE (mean ± std)': f"{mae.mean():.4f} ± {mae.std():.4f}",
        'MSE (mean ± std)': f"{mse.mean():.4f} ± {mse.std():.4f}",
        'RMSE (mean ± std)': f"{rmse.mean():.4f} ± {rmse.std():.4f}",
        'R2 (mean ± std)': f"{r2.mean():.4f} ± {r2.std():.4f}"
    }
    return summary

```

```

[41]: # Résumé des résultats
results = []
results.append(summarize_results(dt_cv, "Decision Tree"))
results.append(summarize_results(rf_cv, "Random Forest (full)"))
results.append(summarize_results(rf_top3_cv, "Random Forest (Top 3 features)"))

# Display results in a table
results_df = pd.DataFrame(results)
print(results_df)

```

	Model	MAE (mean ± std)	MSE (mean ± std)	\
0	Decision Tree	0.6149 ± 0.1826	0.8562 ± 0.3014	
1	Random Forest (full)	0.5208 ± 0.1166	0.5277 ± 0.2003	
2	Random Forest (Top 3 features)	0.5808 ± 0.0912	0.5762 ± 0.1603	

	RMSE (mean ± std)	R ² (mean ± std)
0	0.9131 ± 0.1498	0.7321 ± 0.0873
1	0.7125 ± 0.1418	0.8257 ± 0.0881
2	0.7513 ± 0.1080	0.8106 ± 0.0846

1.4.2 Features importance

```

[42]: # Plot
plt.figure(figsize=(10, 6))
sns.barplot(
    x=feature_importances.values,
    y=feature_importances.index,
    palette="Blues_d"
)
plt.title(" Feature Importances - Random Forest")
plt.xlabel("Importance")

```

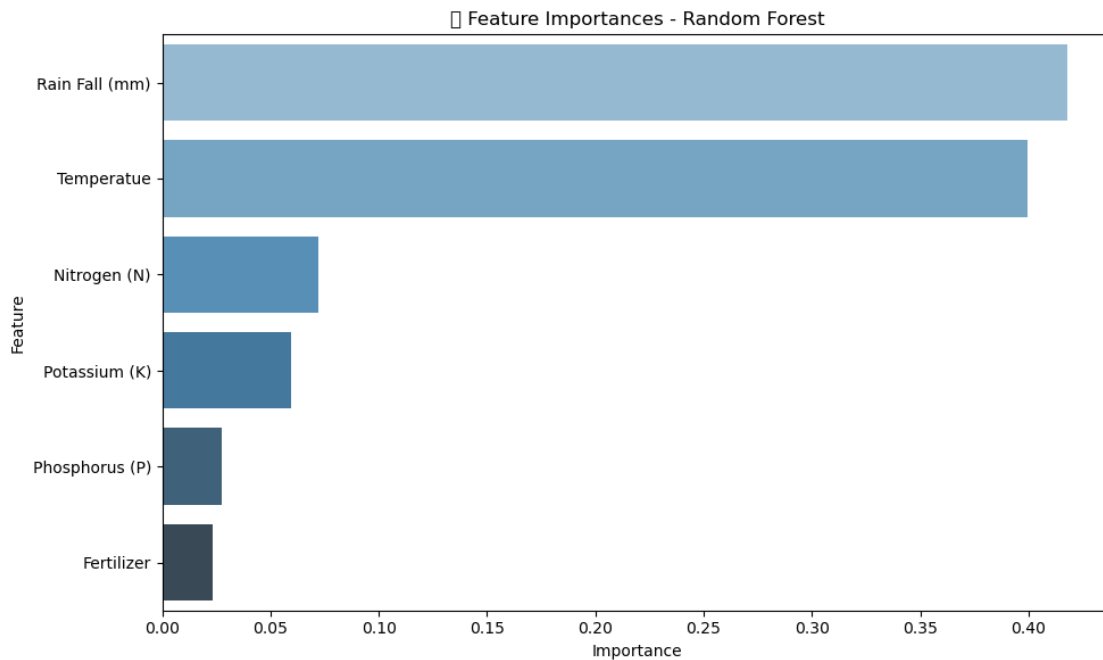
```
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

C:\Users\jeane\AppData\Local\Temp\ipykernel_26692\1556956996.py:11: UserWarning: Glyph 127794 (\N{EVERGREEN TREE}) missing from current font.

```
plt.tight_layout()
```

C:\Users\jeane\anaconda3\Lib\site-packages\IPython\core\pylabtools.py:152: UserWarning: Glyph 127794 (\N{EVERGREEN TREE}) missing from current font.

```
fig.canvas.print_figure(bytes_io, **kw)
```



```
[43]: from sklearn.model_selection import GridSearchCV, KFold
```

```
#parameters for grid search
```

```
param_grid = {
    'n_estimators': [100, 150, 200],
    'max_depth': [10, 15, 20],
    'min_samples_split': [2, 4],
    'min_samples_leaf': [1, 2],
    'max_features': ['sqrt', 'log2']
}
```

```
cv = KFold(n_splits=5, shuffle=True, random_state=42)
```

```
rf = RandomForestRegressor(random_state=42)
```

```
grid = GridSearchCV(
```



```

    estimator=rf,
    param_grid=param_grid,
    scoring='r2',          # ou 'neg_mean_absolute_error', etc.
    cv=cv,
    n_jobs=-1,            # utilise tous les cœurs disponibles
    verbose=2
)

grid.fit(X, y)

print(" Meilleurs hyperparamètres :")
print(grid.best_params_)

print(f"\n Meilleur score R² : {grid.best_score_:.4f}")

```

Fitting 5 folds for each of 72 candidates, totalling 360 fits

Meilleurs hyperparamètres :

```
{'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 1,
'min_samples_split': 4, 'n_estimators': 150}
```

Meilleur score R² : 0.8367

1.4.3 Entraînement du meilleur model optimisé sur le dataset

```
[45]: best_rf_model = grid.best_estimator_
      best_rf_model.fit(X, y)
```

```
[45]: RandomForestRegressor(max_depth=10, max_features='sqrt', min_samples_split=4,
                           n_estimators=150, random_state=42)
```

1.4.4 Comparaison avec les autres models

```
[46]: optimized_rf_cv = cross_validate(best_rf_model, X, y, cv=cv, scoring=scoring)

      results.append(summarize_results(optimized_rf_cv, "Random Forest (Optimized)"))

      results_df = pd.DataFrame(results)
      print(results_df)
```

	Model	MAE (mean ± std)	MSE (mean ± std)	\
0	Decision Tree	0.6149 ± 0.1826	0.8562 ± 0.3014	
1	Random Forest (full)	0.5208 ± 0.1166	0.5277 ± 0.2003	
2	Random Forest (Top 3 features)	0.5808 ± 0.0912	0.5762 ± 0.1603	
3	Random Forest (Optimized)	0.5045 ± 0.1075	0.4799 ± 0.1988	

	RMSE (mean ± std)	R² (mean ± std)
0	0.9131 ± 0.1498	0.7321 ± 0.0873
1	0.7125 ± 0.1418	0.8257 ± 0.0881

```
2  0.7513 ± 0.1080  0.8106 ± 0.0846
3  0.6781 ± 0.1415  0.8367 ± 0.1010
```

1.4.5 Features importances du model optimisé

```
[48]: importances_optimized = pd.Series(
        best_rf_model.feature_importances_,
        index=X.columns
    ).sort_values(ascending=False)

    print(importances_optimized.head(10))
```

```
Rain Fall (mm)    0.305461
Temperatue        0.264859
Nitrogen (N)      0.177664
Potassium (K)     0.111208
Fertilizer        0.099891
Phosphorus (P)    0.040916
dtype: float64
```

1.5 Save the best model

```
[50]: import joblib

    joblib.dump(best_rf_model, 'random_forest_model.pkl')
```

```
[50]: ['random_forest_model.pkl']
```

1.6 Conclusion

From the above analysis in the Exploratory Data Analysis, it is clear that there are two crops in the dataset, whose presence has been confirmed with presence of two definite clusters in the graphs of rainfall, temperature, and crop yield. The dataset has been collected for two different crops. The dataset has signs of proportional relationship between the nutrients and the crop yield. However, the relationship between the crop yield and the other columns is not directly proportional. This could be due to the other factors such as soil type, weather conditions, and crop breed. On the whole, the dataset is not much complex to make concrete decisions based on the graphs only.

We first trained baseline models using a Decision Tree Regressor and a Random Forest Regressor. Among these, the Random Forest model demonstrated superior performance in terms of R^2 , RMSE, MAE, and MSE.

To further improve accuracy, we applied GridSearchCV to optimize the hyperparameters of the Random Forest model. The optimized model achieved the highest R^2 score, confirming the effectiveness of hyperparameter tuning.

Additionally, we analyzed feature importances and experimented with a simplified version of the model using only the top 3 most influential features. While the reduced model showed competitive results, it was slightly outperformed by the full optimized Random Forest, indicating that other features still contribute meaningful variance.

Overall, the optimized Random Forest Regressor stands out as the best-performing model for this task, balancing predictive power and generalization.